Referencia TAC a MIPS

Repositorio: https://github.com/Jonialen/proyecto-compis

Jonathan Diaz Luis Padilla

Índice

1. Recorrido mínimo reproducible 1
2. Convenciones comunes 2
3. Mapa de instrucciones 2
4. Llamada con temporales vivos 3
5. Capturas mutables 3
6. Rechazo sin mutar la colección 3
7. Excepciones como valores retenidos 3
8. Syscalls y límites de interpretación 4

Esta referencia conecta los opcodes de `tac.Program` con el ensamblador y el runtime generados. Los fragmentos explican el contrato; los nombres físicos y desplazamientos dependen de cada programa. Para obtener el archivo completo usá la CLI MIPS, no copies un fragmento aislado como si incluyera el runtime.

1. Recorrido mínimo reproducible

Creá `demo.cps` con:

```
print(2 + 3);
```

Desde la raíz del repositorio:

```
go run ./cmd/compiscript-tac demo.cps
go run ./cmd/compiscript-mips demo.cps > /tmp/demo.asm && \
  bash scripts/mars.sh /tmp/demo.asm
```

El primer comando muestra el TAC:

```
t0:integer = 2:integer + 3:integer
print t0:integer
```

El segundo produce y ejecuta ensamblador; stdout debe ser 5 seguido de salto de línea. MARS agrega una línea vacía a su stream de mensajes stderr. No se evalúa 2 + 3 en Go para fabricar esa salida: el archivo contiene la operación y MARS la ejecuta.

Fragmento conceptual de la operación emitida

```
# s3 y s4 apuntan a cajas de los operandos; los tags ya se validaron.
l.d   $f0, 8($s3)
l.d   $f2, 8($s4)
add.d $f4, $f0, $f2
li    $a0, 16
jal   rt_alloc
li    $t0, 1
sw    $t0, 0($v0)
s.d   $f4, 8($v0)
# El puntero v0 se guarda después en el home canónico de t0.
```

El hecho de usar `add.d` no borra la distinción del lenguaje: la caja lleva etiqueta entera 1 o flotante 2. Las guardas y conversiones trabajan con esa etiqueta y el payload binary64.

2. Convenciones comunes

- Un operando literal se materializa como dato etiquetado; un símbolo/temporal se carga desde su home o celda de entorno.
- Leer un binding no declarado o declarado sin valor produce un fallo. `null` es una caja válida y distinta de esos estados.
- El destino se escribe después de leer sus operandos. Esto permite que un temporal reciclado sea a la vez un operando y el destino.
- Las etiquetas TAC se califican físicamente por procedimiento para no colisionar entre funciones.
- Los helpers devuelven punteros de valores por `$v0` o el resultado específico documentado; las ubicaciones canónicas preservan valores vivos.
- Un opcode, operando o layout inválido causa rechazo atómico durante la generación. Las guardas de valores del lenguaje se ejecutan en MIPS y pueden llegar a catch.

3. Mapa de instrucciones

Familia TAC	Traducción o helper	Regla importante
`copy`	Cargar puntero y guardarlo en el destino	Los alias de objetos/listas conservan identidad
`declare`	Guardar el sentinel de binding sin valor	No inicializa artificialmente con cero/null
`+`, `-`, `*`, `/`, `%`	Carga de payload, FPU y caja de resultado; suma string mediante `rt_concat`	División/resto por cero fallan; `%` no trunca el cociente a 32 bits
`neg`, `!`, `i2f`	Negación numérica/booleana o caja flotante nueva	Promover no cambia un valor compartido
`==`, `!=`	`rt_equal` más selección de booleano	Comparación por contenido escalar, identidad de referencias y tupla de función
`<`, `<=`, `>`, `>=`	Comparaciones FPU y caja booleana	Las operaciones guardadas validan tags efectivos
`print`	`rt_print` y salto de línea	Strings por longitud; listas con límites; IDs de referencia estables
`label`, `jump`, `branchTrue`	Etiquetas físicas, saltos y guarda booleana	No se convierte cualquier entero en booleano
`call`, `return`	Vector por pila, `jal`, epílogo y `jr`	Frame propio, preservación y resultado por `$v0`
`enterEnvironment`	Nueva instancia de entorno en heap	Se ejecuta en cada entrada, no solo una vez por layout
`makeClosure`	Caja con código, entorno concreto y receptor cero	Las capturas sobreviven al retorno
`callIndirect`	Validación de callee/firma, `jalr`	Selección congelada antes de efectos posteriores en argumentos
`newObject`	Caja extendida y entornos de clase/ancestros	No vuelve a ejecutar inicializadores ya representados en TAC
`loadField`, `storeField`	Offset validado y acceso a celda del objeto	Guarda de null/receiver y política readonly
`bindMethod`	Caja de función ligada	Dispatch efectivo por miembro, o inicializador explícito
`newList`, `listLength`	Asignación de lista o caja entera de longitud	Elementos evaluados/congelados en orden
`loadIndex`, `storeIndex`	`ref_index`, acceso y comprobaciones de tipo	Finitud, integridad y límites antes de estrechar el índice
`checkValue`	`ref_check` con descriptor	Guarda nominal/shape/conversión sin perder identidad
`dynamicMember`, `dynamicStore`	Selección según clase efectiva y guarda	Campo ausente/readonly/tipo incompatible fallan

Familia TAC	Traducción o helper	Regla importante
`pushHandler`, `popHandler`, `catchException`	Registro dinámico, restauración y binding de excepción	Retirar handler antes del catch; respetar limpiezas TAC

La información `Guarded` solicita comprobaciones del valor real cuando la semántica conserva información desconocida. `TypeError` en ese contexto es metadata de información ausente, no un nuevo tipo público del lenguaje.

4. Llamada con temporales vivos

```
function fib(n: integer): integer {
    if (n <= 1) return n;
    else return fib(n - 1) + fib(n - 2);
}
print(fib(8));
```

El primer resultado recursivo debe sobrevivir a la segunda llamada. El backend lo conserva en un home del frame del caller, no en un `$t` que el siguiente `jal` pueda destruir. El resultado esperado es 21.

Los argumentos se pasan como vector de punteros. La función copia ese vector a sus homes antes de ejecutar otra operación. Esto también cubre diez o más valores vivos: usar registros `$a` para el puntero del vector no limita el lenguaje a cuatro parámetros.

5. Capturas mutables

```
function zero(): integer { return 0; }
let saved = zero;
function install(seed: integer) {
    let n = seed;
    function next(): integer { n = n + 1; return n; }
    saved = next;
}
install(10);
print(saved());
print(saved());
```

Salida esperada:

```
11
12
```

La caja de `saved` retiene la instancia del entorno de `install`. `n` no reside únicamente en un frame que ya se liberó; dos llamadas a la misma clausura observan la misma celda. Otra llamada a `install` produce una instancia diferente.

6. Rechazo sin mutar la colección

```
let values = [1];
function put(xs, value) { xs[0] = value; }
try { put(values, true); }
catch (error) { print("rejected"); }
print(values);
```

Salida esperada:

```
rejected
[1]
```

La escritura desconocida comprueba el contrato de la asignación original antes del store. El handler recupera el control sin transformar el fallo en una escritura parcial.

7. Excepciones como valores retenidos

```
try { let bad = 1 / 0; }
catch (error) { print([error]); }
```

Salida esperada: `[division by zero]` y salto de línea. La excepción usa etiqueta 9 y puede quedar en una lista o captura; no se convierte en un string sin identidad. Este caso encontró la omisión de descriptor corregida antes del cierre de la aceptación.

8. Syscalls y límites de interpretación

Servicio MARS	Uso
1	Impresión entera cuando el double es integral y cabe en 32 bits
3	Impresión double para los demás números
4	Textos internos NUL-terminados y mensajes ASCII del runtime
9	Asignación de heap, con alineación aplicada por el helper
10	Finalización normal del programa principal
11	Caracteres de formato de referencias/listas
15	Bytes string/excepción por longitud en stdout
17	Terminación con código de error cuando el fallo no tiene handler

El wrapper MARS usa `nc me ae1 se2`: omite el banner, dirige mensajes del simulador a stderr y fija los códigos de error de ensamblado/simulación. No activa delayed branching. Una pseudoinstrucción aceptada por MARS o un helper propio no implica compatibilidad con cualquier ensamblador o ABI externo.

La arquitectura fija offsets y registros; la guía distingue compilación, ensamblado y ejecución; la evidencia delimita qué casos se probaron realmente.